DevDocs — Document Explorer

1. Purpose

The **Document Explorer** is the part of DevDocs where users can browse, search, inspect, and manage the files belonging to a project.

It should help developers answer:

> **"What files are inside this project, where are they, and what is their current ingestion state?"**

The Document Explorer is **not an IDE**.

It is primarily a project understanding and navigation tool.

---

2. Project File Tree

The Document Explorer will display the project's actual folder structure using a collapsible file/folder tree.

Example:

```
my-project/
├── 📁 backend
│   ├── 📄 main.py
│   └── 📄 auth.py
├── 📁 frontend
│   └── 📄 App.jsx
└── 📄 README.md
```

The structure shown to the user should preserve the original relative paths from the uploaded project.

Ignored Files and Directories

Files and directories that are ignored during ingestion should also be completely hidden from the Document Explorer.

Examples:

```
.git/
node_modules/
venv/
.venv/
```

```
__pycache__/
dist/
build/
.env
```

This creates a consistent rule:

```
Ignored
   ↓
Not processed
   ↓
Not stored as project knowledge
   ↓
Not displayed in Document Explorer
   ↓
Cannot appear in RAG
```

The user therefore sees only the meaningful project files.

---

3. Floating File Viewer

When the user clicks a file, it should open inside a **small floating internal viewer** rather than navigating away from the project.

Example:

```
┌──────────────────────────────────────────────┐
│ auth.py                                  ×   │
├──────────────────────────────────────────────┤
│  1  from fastapi import Depends              │
│  2                                           │
│  3  def authenticate_user(token):            │
│  4      ...                                  │
│  5      ...                                  │
│                                              │
└──────────────────────────────────────────────┘
```

The project file tree remains visible behind/alongside the viewer.

Why a floating viewer?

It allows the developer to:

- quickly inspect a file
- keep the project structure visible
- switch between files easily

- remain inside the current project
- eventually jump directly to cited lines

The viewer is **read-only in V1**.

DevDocs should not become an IDE.

---

4. Different File Renderers

Different file types should be rendered appropriately inside the same floating viewer.

The architecture can use one main viewer that selects the appropriate renderer:

```
FloatingFileViewer
    ├── CodeRenderer
    ├── MarkdownRenderer
    ├── PDFRenderer
    ├── TableRenderer
    └── TextRenderer
```

V1 Rendering

File type	Viewer
`.py`	Code viewer + syntax highlighting + line numbers
`.java`	Code viewer + syntax highlighting + line numbers
`.js`	Code viewer + syntax highlighting + line numbers
`.jsx`	Code viewer + syntax highlighting + line numbers
`.ts`	Code viewer + syntax highlighting + line numbers
`.tsx`	Code viewer + syntax highlighting + line numbers
`.json`	Code/text viewer + syntax highlighting
`.yaml` / `.yml`	Code/text viewer + syntax highlighting
`.md`	Rendered Markdown
`.txt`	Plain text + line numbers
`.pdf`	PDF viewer with page navigation
`.docx`	Document renderer
`.html`	Source-code viewer, not executable HTML
`.csv`	Table viewer

The viewer will **not** provide:

- Editing
- Saving changes
- Terminal
- Code execution
- Debugging
- Extensions

DevDocs is for **understanding and navigating a project**, not modifying it.

---

5. Future Citation → File Navigation

The floating viewer also gives us an important future capability.

For example:

```
Citation
   ↓
backend/auth/jwt.py
Lines 42–57
   ↓
Floating viewer opens
   ↓
Scroll to line 42
   ↓
Highlight lines 42–57
```

This will make RAG citations much more useful for developers.

The basic viewer therefore needs to support line-based navigation for code files.

---

6. File Search

The Document Explorer will contain a search bar at the top.

The purpose of V1 file search is:

> **“Where is this file?”**

It is not intended to answer:

> **“Where is this concept implemented?”**

That second question belongs to RAG/chat.

Search Behavior

Search starts after the user enters at least **2 characters**.

Example:

```
🔍 jwt
```

Results dynamically appear while typing:

```
📄 jwt.py
   backend/auth/

📄 jwt_service.py
   backend/services/

📄 jwt.js
   frontend/utils/
```

Rules

- Minimum 2 characters
- Case-insensitive
- Dynamic filtering as the user types
- Search filename + relative path
- Do not search file contents in V1
- Show the filename
- Show the relative path below it
- Clicking a result opens the floating file viewer
- If there are no matches:

```
No files found
```

Content search can be considered as a future enhancement.

---

7. Individual File Actions

The Document Explorer should provide useful actions at the individual file level.

V1 Supported Actions

Action	V1
Open file	✅

Action	V1
Search file	✅
View ingestion status	✅
Replace/re-upload file	✅
Delete file	✅
Edit file inside DevDocs	❌
Rename file inside DevDocs	❌
Move file inside DevDocs	❌

---

8. Individual File Deletion

When a user deletes a file, DevDocs should ask for confirmation.

After confirmation:

```
File
 ↓
Uploaded file removed
 ↓
File metadata removed
 ↓
Chunks removed
 ↓
Embeddings/vector records removed
```

Deleting an individual file must **not** delete:

- The project
- Other project files
- Conversations
- Conversation messages

The deletion is limited to that file and its associated ingestion data.

---

9. Individual File Replacement

A user can replace/re-upload an individual file.

Example:

```
Existing:
backend/auth.py
```

The user uploads a new version.

DevDocs calculates its SHA-256 hash and compares it with the existing file.

```
New file
   ↓
Calculate SHA-256
   ↓
Compare with existing hash
```

Same hash

```
Same content
   ↓
Skip processing
```

Different hash

```
Different content
   ↓
Remove old chunks
   ↓
Process new file
   ↓
Chunk
   ↓
Embed
   ↓
Store new chunks
```

This reuses the same incremental ingestion system used for full-project re-upload.

---

10. Conversations After File Deletion

Deleting a file should not delete old conversations.

For example:

```
Conversation:
"How does authentication work?"
```

may contain a citation to:

```
backend/auth.py
```

If that file is later deleted, the old conversation remains stored.

However, the old citation may no longer be clickable/current because its source file no longer exists.

This is preferable to deleting historical conversation data.

---

11. Why DevDocs Does Not Support File Editing

DevDocs should not provide an internal code editor in V1.

The product is focused on:

- Understanding
- Searching
- Navigating
- Explaining
- Documenting
- Asking questions about a project

It is not intended to replace:

- VS Code
- IntelliJ
- Cursor
- Other development environments

Keeping files read-only also keeps the architecture considerably simpler.

---

12. File-Level Ingestion Status

Each file should have a simple ingestion status.

The four V1 statuses are:

Ready

```
📄 auth.py
   ✓ Ready
```

Meaning:

- File was successfully parsed
- Content was chunked
- Embeddings were generated
- File is available for RAG

Processing

```
📄 payment.py
   ⟳ Processing...
```

Meaning the file is currently being ingested.

The file can still be viewed, but RAG should not depend on it until ingestion is complete.

Failed

```
📄 config.py
   × Failed
```

The file could not be successfully processed.

The user should be able to retry or replace it.

Pending

```
📄 database.py
   ○ Pending
```

The file is waiting to be processed.

This is useful when multiple files are being ingested.

---

13. Failed File Handling

A failed file should **not disappear** from the project.

For example:

```
backend/
├── main.py       ✓ Ready
├── auth.py       ✓ Ready
├── payment.py    ⟳ Processing
└── config.py     × Failed
```

The user can:

- See the file
- Open/view the file
- See that ingestion failed
- Retry ingestion
- Replace the file

The file simply isn't available to RAG until successful ingestion.

Failed File UI

Clicking the failed status can show a small message:

```
┌──────────────────────────────┐
│ config.py                    │
│                              │
│ × Failed to process          │
│                              │
│ DevDocs couldn't process this│
│ file.                        │
│                              │
│ [Retry]       [Replace File] │
└──────────────────────────────┘
```

The main UI should show a **simple user-friendly error**.

Technical stack traces and detailed backend errors should remain in backend logs.

---

14. Retry Flow

When the user selects **Retry**:

```
Failed
  ↓
Pending
  ↓
Processing
  ↓
Ready
```

If processing fails again:

```
Processing
    ↓
Failed
```

The latest ingestion attempt becomes the current state.

---

15. Replace Failed File

If the problem is with the file itself, the user can replace it.

Example:

```
config.py
   × Failed
       ↓
User uploads corrected config.py
       ↓
Calculate SHA-256
       ↓
Process
       ↓
Ready
```

Once the file becomes `Ready`, it automatically becomes available to RAG.

---

16. Partial Ingestion

A single failed file should **not make the entire project unusable**.

For example:

```
Project
│
├── main.py       → Ready
├── auth.py       → Ready
├── payment.py    → Processing
└── config.py     → Failed
```

RAG can use:

```
main.py
auth.py
```

but cannot use:

```
payment.py
config.py
```

This allows developers to start using DevDocs even when some files are unavailable.

Core Principle

> **A single bad file should not make the entire project unusable.**

---

17. Project-Level Ingestion Status

The project itself also has an overall ingestion status.

The UI should remain minimalistic.

During ingestion

```
⟳ Processing
```

We do **not** need to permanently display something like:

```
12 / 20 files ready
```

because that makes the interface unnecessarily busy.

Detailed file-level statuses already provide the required information.

When complete

```
✓ Ready
```

No `20/20 files ready` indicator is necessary.

---

18. Chat Availability During Ingestion

Chat behavior depends on whether any usable project knowledge is available.

Zero files ready

Chat should be disabled.

There is currently no processed project knowledge that RAG can use.

Example:

```
⟳ DevDocs is still processing this project.

Chat will be available once some project files
are ready.
```

Some files ready

Chat should be available.

For example:

```
15 files ready
5 files processing
```

The user can already ask questions.

RAG searches only the ready files.

If the user asks something that likely depends on a file that is still processing, DevDocs should not pretend that it has access to that file.

Example response:

> I couldn't fully answer this yet because `payment.py` is still being processed.

All files ready

Chat works normally.

```
✓ Ready
```

Some files failed

Chat remains available using successfully processed files.

```
18 files ready
2 files failed
```

All files failed

Chat should remain unavailable because there is no usable project knowledge.

---

19. Project-Level Status Rules

Project state	Chat
0 files ready	❌ Disabled
Some files ready + others processing	✅ Allowed
All files ready	✅ Allowed
Some files failed	✅ Allowed
All files failed	❌ Disabled
Processing completed with errors	✅ Allowed

---

20. RAG Must Respect Ingestion Status

The frontend status should not be the authority for RAG.

The backend must enforce the rule.

Conceptually:

```
Project
   ↓
Files
   ↓
Only READY files
   ↓
Chunks
   ↓
Vector search
   ↓
LLM
```

Processing or failed files must never accidentally enter the retrieval context.

The application/retrieval layer is responsible for enforcing this boundary.

---

21. Project Ingestion Continues in the Background

Ingestion should continue even if the user leaves the current project page.

Example:

```
User uploads project
       ↓
Ingestion starts
       ↓
User navigates to another project
       ↓
Backend continues processing
```

The frontend should not be responsible for keeping ingestion alive.

---

22. Global Ingestion Indicator

Since the user may leave the project page while ingestion continues, DevDocs can show a small global indicator in the main application UI.

Example:

```
⟳
```

Hovering over it shows:

```
┌──────────────────────────────┐
│ Processing project           │
│ E-Commerce API               │
└──────────────────────────────┘
```

This provides awareness without taking up significant UI space.

When ingestion finishes, the indicator disappears.

---

23. Upload/Re-upload During Active Ingestion

To keep V1 simple, the user should **not be allowed to upload or re-upload a project while ingestion is already running**.

This avoids unnecessary complexity around:

- Multiple ingestion jobs
- Conflicting project states
- Concurrent uploads
- Merging ingestion operations
- Race conditions
- Complicated job queues

The simple rule is:

```
Ingestion running
       ↓
Upload/re-upload disabled
       ↓
Ingestion finishes
       ↓
Upload/re-upload available again
```

Individual file replacement should follow the same restriction while the relevant ingestion operation is active.

---

24. Progress Calculation

V1 does not need complicated progress tracking.

We do not need to expose:

```
Parsing: 70%
Chunking: 40%
Embedding: 20%
```

Instead, progress is represented simply through file-level statuses:

```
✓ Ready
◌ Processing
○ Pending
× Failed
```

This is enough to communicate the project's state while keeping the interface minimal.

---

25. Crash Recovery

Sophisticated ingestion recovery is **out of scope for V1**.

For now, the ingestion system should work normally within the application's current runtime.

Future versions can introduce:

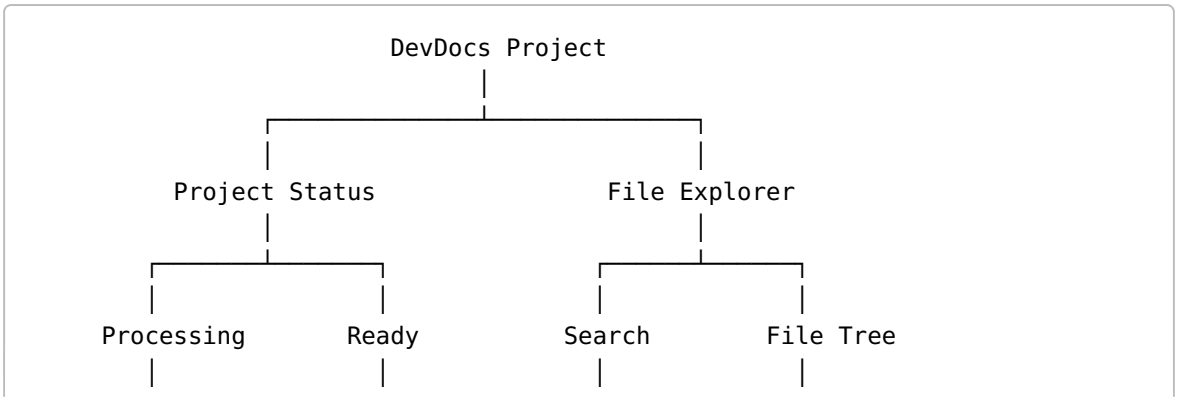
- Job persistence
- Automatic retry
- Resumable ingestion
- Background workers
- Queue-based processing
- Recovery after server crashes

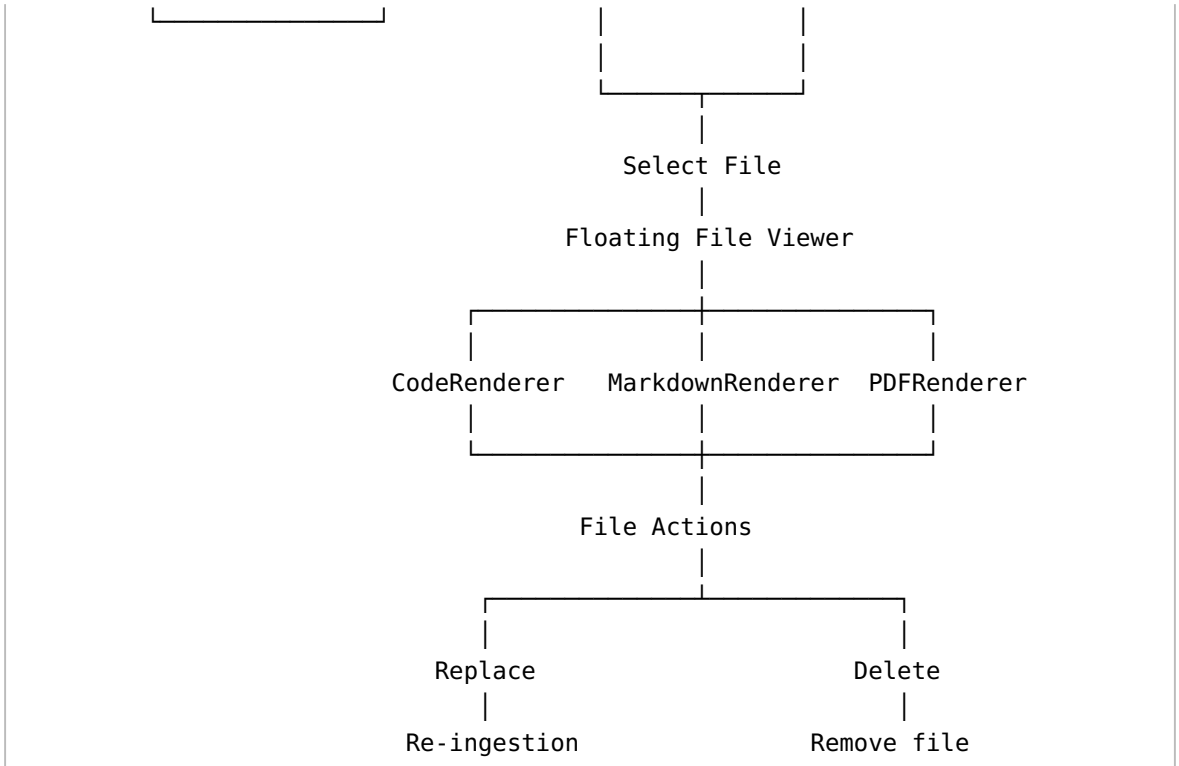
These are intentionally deferred.

---

26. Final Document Explorer Architecture

The overall Document Explorer can therefore be visualized as:





---

27. Relationship With RAG

The Document Explorer and RAG serve different purposes.

Document Explorer

Answers:

> **“What is in my project?”**

Examples:

- Where is `auth.py`?
- What files are under `backend/`?
- What is the current ingestion status?
- Let me inspect `payment_service.py`.

RAG / Chat

Answers:

> **“Help me understand my project.”**

Examples:

- How does authentication work?

- Where is payment processing implemented?
- Explain the order creation flow.
- Why is this service calling the database?

Together:

```
Document Explorer
       ↓
Navigate / Inspect
       +
Chat / RAG
       ↓
Understand / Explain
```

---

28. V1 Scope

The V1 Document Explorer includes:

- Collapsible project file tree
- Original project folder structure
- Ignored files/directories hidden
- Floating file viewer
- File-type-specific rendering
- Read-only file viewing
- Filename/path search
- Minimum 2-character search
- Case-insensitive search
- Individual file opening
- Individual file deletion
- Individual file replacement/re-upload
- File ingestion statuses
- Project-level ingestion status
- Failed-file handling
- Retry failed files
- Partial ingestion
- RAG exclusion for failed/processing files
- Background ingestion
- Global ingestion indicator
- Upload/re-upload restriction during active ingestion
- Simple file-level progress

---

29. Future Enhancements

The following are intentionally deferred:

- Content-based file search
- Clickable RAG citations
- Automatic jump to cited lines
- Line highlighting
- Advanced code navigation
- File editing
- Rename/move files inside DevDocs
- AST-aware code navigation
- Dependency graph
- Framework detection
- Technology detection
- GitHub integration
- Automatic repository synchronization
- Branch-aware projects
- Project version history
- Background job queues
- Sophisticated crash recovery
- Resumable ingestion
- Detailed ingestion dashboard
- Fine-grained parsing/chunking/embedding progress

---

30. Core Design Principles

1. Keep the interface minimal

Don't expose internal processing details unless they help the developer.

2. Don't make DevDocs an IDE

Files are for inspection and understanding, not editing or execution.

3. A failed file shouldn't break the project

Support partial ingestion.

4. Only ready knowledge enters RAG

Processing and failed files must be excluded.

5. Background work should remain background work

Users shouldn't have to stay on the project page while ingestion runs.

6. Use the file tree for navigation

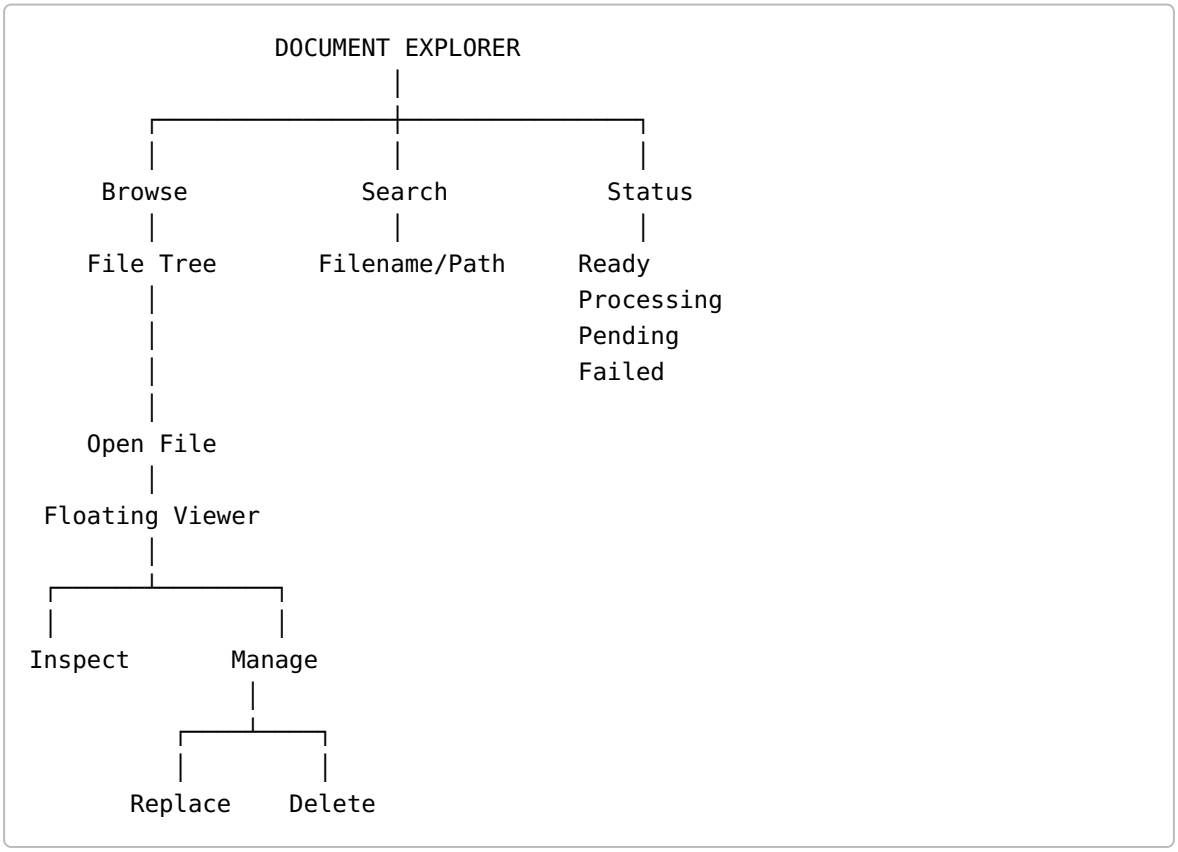
Use RAG for understanding.

7. Keep V1 simple

Avoid concurrency, recovery systems, detailed progress tracking, and other complexity until they are actually needed.

Final Mental Model

The Document Explorer is essentially:



And its relationship with the rest of DevDocs is:

```
Project
   │
   ├── Document Explorer
   │       ├── Browse
   │       ├── Search
   │       ├── Inspect
   │       └── Manage files
   │
   ├── Ingestion
   │       ├── Parse
```

```
    │       ├── Chunk
    │       ├── Embed
    │       └── Track status
    │
    └── Chat / RAG
            ├── Retrieve ready project knowledge
            ├── Understand the question
            ├── Generate explanation
            └── Cite project sources
```

Core idea:

> **Document Explorer lets developers see and navigate their project. RAG lets DevDocs understand and explain that project.**